

Kernel Module Development on WSL2

From Zero to Hello World — No VirtualBox

1 Why a Custom Kernel?

The stock WSL2 kernel is a pre-built binary with no matching headers or build system. Building a module against it fails; even if it compiles, loading it fails:

```
make: *** /lib/modules/$(uname -r)/build: No such file or directory
insmod: ERROR: could not insert module ./hello.ko: Invalid module format
```

Fix: build your own kernel from the same source tree, boot WSL with it, then build/load modules against it.

2 Prerequisites

```
wsl --install
```

Restart, open Ubuntu, create a user, then:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y build-essential flex bison libssl-dev libelf-dev bc dwarves git
```

3 Get the Kernel Source

```
cd ~
git clone --depth=1 -b linux-msft-wsl-6.18.y \
  https://github.com/microsoft/WSL2-Linux-Kernel.git
cd WSL2-Linux-Kernel
```

4 Prepare the Kernel Tree

Order matters here. `mrproper` wipes `.config`, so it must run *first*.

```
make mrproper
cp Microsoft/config-wsl .config
make olddefconfig
make -j$(nproc) modules_prepare
```

`modules_prepare` alone does **not** create a usable `Module.symvers`. Run the full modules build (20–60 min) to generate the real symbol table:

```
make -j$(nproc) modules
ls -lh Module.symvers
wc -l Module.symvers
```

5 Build the Custom Kernel (bzImage)

```
make -j$(nproc)
ls -l arch/x86/boot/bzImage
```

6 Boot WSL With Your Kernel

```
ls /mnt/c/Users/
cp arch/x86/boot/bzImage /mnt/c/Users/YOUR_USERNAME/
```

Create C:\Users\YOUR_USERNAME\.wslconfig:

```
[wsl2]
kernel=C:\\Users\\YOUR_USERNAME\\bzImage
```

Restart WSL from PowerShell, then verify:

```
wsl --shutdown
# reopen Ubuntu, then:
uname -r
cat /proc/version
```

A trailing + on the version string confirms your own build is running.

7 Hello World Module

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>

MODULE_LICENSE("GPL");

static int __init hello_init(void) {
    printk(KERN_INFO "hello: module loaded - Hello!\n");
    return 0;
}

static void __exit hello_exit(void) {
    printk(KERN_INFO "hello: module unloaded - Bye!\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

```

MAKEFILE
obj-m += hello.o
all:
    make -C ~/WSL2-Linux-Kernel M=$(PWD) modules
clean:
    make -C ~/WSL2-Linux-Kernel M=$(PWD) clean

```

8 Build and Load

```

cd ~/KERNEL_DEV/Module_files
make clean
make
sudo insmod ./hello.ko
lsmod | grep hello
dmesg | tail -5
sudo rmmod hello
dmesg | tail -5

```

9 Errors, Decoded

- **build: No such file or directory** — no prepared source tree yet (Section 4–5).
- **Invalid module format** — module built against the prepared tree, but WSL still boots the stock kernel. Boot your own kernel first (Section 6–7).
- **undefined symbols** (`_printk`, `module_layout`...) — `Module.symvers` is incomplete; only `modules_prepare` was run. Run the full `make modules`.
- **File exists on insmod** — module is already loaded; check with `lsmod`, not a real error.

10 What Not to Do

- `touch Module.symvers` — an empty file still fails `modpost`.
- `make KBUILD_MODPOST_WARN=1` — hides unresolved symbols instead of fixing them; avoid for real driver work.

11 Quick Reference

```

cd ~ && git clone --depth=1 -b linux-msft-wsl-6.18.y \
    https://github.com/microsoft/WSL2-Linux-Kernel.git
cd WSL2-Linux-Kernel
make mrproper
cp Microsoft/config-wsl .config
make olddefconfig
make -j$(nproc) modules_prepare
make -j$(nproc) modules
make -j$(nproc)
cp arch/x86/boot/bzImage /mnt/c/Users/YOUR_USERNAME/
# set .wslconfig, then: wsl --shutdown
cd ~/KERNEL_DEV/Module_files && make clean && make
sudo insmod ./hello.ko && dmesg | tail -5

```